DATE

2023 PYQ

Ans-1) a) The maximum ~~depth~~ number of nodes in a binary tree with depth d is 2^d.

b) Output after code execution is : 90

c) Initial queue : a b c d
Final queue : d c b a
Steps : dequeue() // b c d
dequeue() // c d
dequeue() // d
enqueue(c) // d c
enqueue(b) // d c b
enqueue(a) // d c b a

d) Given univariate polynomial : $5x^3 - 6x^2 - 3x + 2$
We will store the coefficient of each term and degree can be ~~four~~ evaluated from position of the ~~of~~ coefficient.

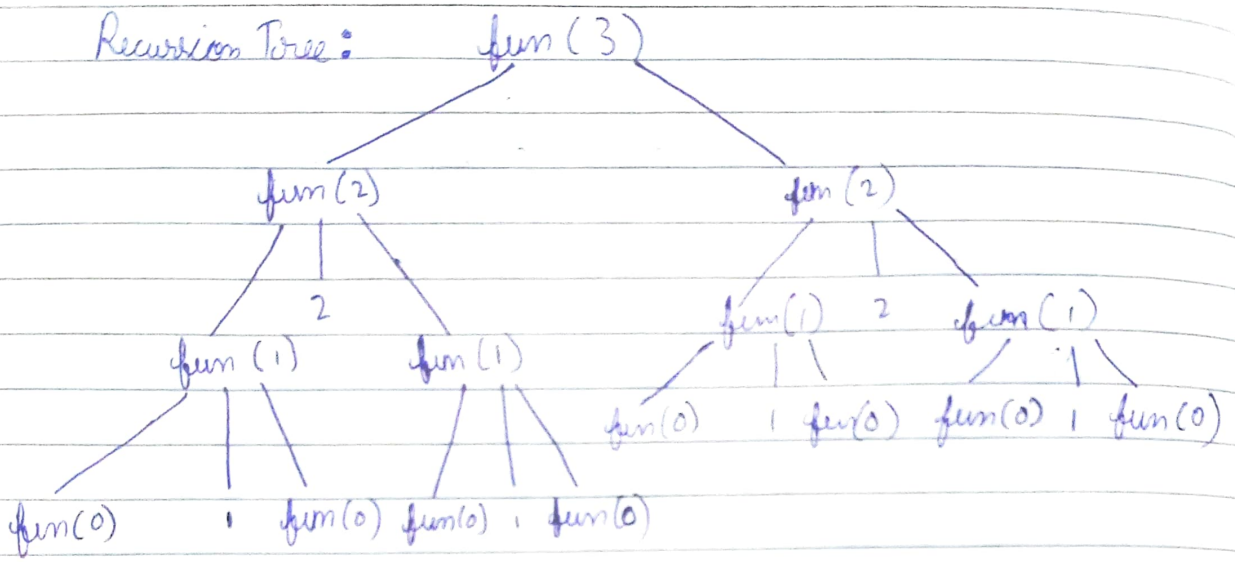
coefficients =	[2] →	[-3] →	[-6] →	[5]
degrees/ position =	0	1	2	3

e) Function to find minimum node element of a Binary search Tree.

```
int findMin (Node* root) {
    if (root = nullptr) {
        cout << "Tree is empty";
        return -1; }
    while (root -> left != nullptr)
        root = root -> left;
    return root -> data;
}
```

Ans-g) Output of the code will be: 1 2 1 3 1 2 1

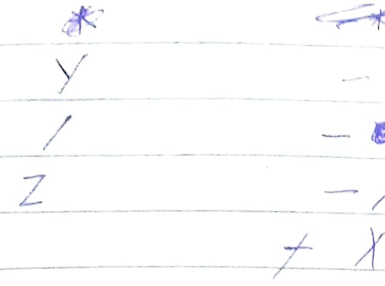
Recursion Tree:



Ans-h) Expression: (A+B)*(C-D)/F - X*Y/Z
Convert to postfix.

Symbol	Stack	Expression
(	(	
A	(	A
+	(+	A
B	(+	AB
)	~~(+~~	AB+
*	*	AB+
(	*(	AB+
C	*(	AB+C
-	*(-	AB+C
D	*(-	AB+CD
)	~~*(-~~	AB+CD-
/	~~*(~~ /	AB+CD*
F	/	AB+CD*F
-	~~/~~ -	AB+CD*F/
X	-	AB+CD*F/X
*	-*	AB+CD*F/X

DATE



AB+CD*F/XY
AB+CD*F/XY*
AB+CD-*F/XY*Z
AB+CD-*F/XY*Z/-

Postfix Expression: AB+CD-*F/XY*Z/-

Answer - i) base address = 1000
Size of each element = 4
array = A[5][10]
The address of A[3][5] using row-major

= 1000 + [(3×10) + 5]×4
= 1000 + 35×4
= 1000 + 140 = 1140

The address of A[3][5] using column major

= 1000 + [(5×5) + 3]×4
= 1000 + 28×4
= 1000 + 112 = 1112

Ans - 2) (a) The main advantage of using doubly linked list over singly linked list is that it allows bi-directional traversal. This feature can be used to access previous node of in O(1) time. — in singly linked list it takes O(n) time to access the previous node. This allows faster insertion and deletion also.

DATE

Function to remove duplicates elements from a sorted ~~linked~~ doubly linked list.

```
void removeDuplicates(Node* head) {
    if (head == nullptr || head->next == nullptr) {
        return;
    }

    Node* current = head;
    while (current != nullptr && current->next
                                != nullptr) {
        if (current->data == current->next->data) {

            Node * temp = current->next;
            current->next = temp->next;
            current->next->prev = current;

            delete temp;
        }
        else {
            current = current->next;
        }
    }
}
```

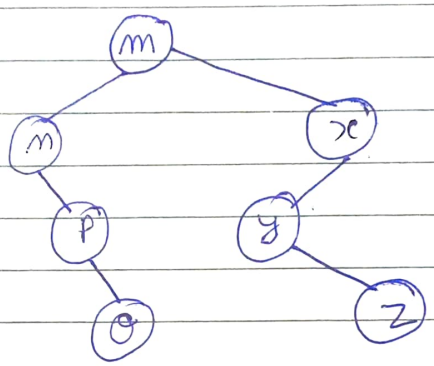
DATE

Ans-2) (b) Given:

Preorder: m n o p x y z

Inorder: n p o m y z x

Binary Tree:



Post order Traversal: o p n z y x m

Ans-3) (a) function to count all occurances of an element in an integer type array.

```
int countAccurances (int el, int arr[]) {
    int lenght = sizeof(arr) / sizeof(arr[0]);
    int count = 0;
    for (int i=0; i <lenght; i++) {
        if ( arr[i] == el )
            count++;
    }
    return count;
}
```

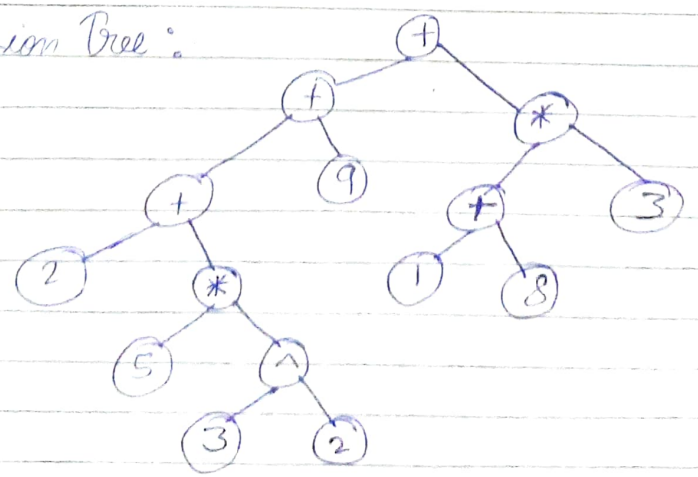
Ans-3 (b) Expression: 43 * 2/182 ^ ^ + 2-3+

Symbol	Stack (→ Top)	Comment
4	4	
3	4, 3	
*	12	4 × 3 = 12
2	12, 2	
/	6	12/2 = 6
1	6, 1	
8	6, 1, 8	
2	6, 1, 8, 2	
^	6, 1, 64	$8^2 = 64$
^	6, 1	$1^{64} = 1$
+	7	6 + 1 = 7
2	7, 2	
-	5	7 - 2 = 5
3	5, 3	
+	8	5 + 3 = 8

Result = 8

Ans (c) Expression: 2 + 5 * 3 ^ 2 + 9 + ((1+8) * 3)

Expression Tree:



Ans-4

DATE

Ans-4 Ans(a)

Program for Next Greater Element:

```
void nextGreaterElement (int arr[], int n) {
1   [ int result[n];
      stack<int> s;

2   [ for (int i=0; i<n; i++) {
        result[i] = -1;
    }

    [ for (int i=0; i<n; i++) {

        while (! s.empty() && arr[i] > arr[s.top()])
        {
3           int index = ~~s.otop~~ s.top();
            s.pop();
            result[index] arr[i];
        }
        s.push(i);
    }

4   [ for (int i=0; i<n; i++) {
        cout << arr[i] << "" --> "" << result[i]
                                    << endl;
    }

}
```

NGE for [4, 5, 2, 25] is : [5, 25, 25, -1]

~~Time complexity~~ :

DATE

Time Complexity:

Code block 1 takes C time

Code block 2 takes C.n time

Code block 3 takes at most 2 C n time because inner while loop runs for at most two time for each element.

Code block 4 takes C.n time.

Total time = $C + C.n + 2.Cn + Cn$

$\leq C'n$

$O(n)$

Ans-6) Recursive function to calculate n^{th} fibonacci number

```
int fib (int n) {
    if (n == 0)
        return 0;
    else if (n == 1)
        return 1;
    else
        return fib(n-1) + fib(n-2);
}
```

DATE

Ans-5 Ans a) Queue ADT using two stacks

```
class Queue {
    stack <int> S1, S2;
public:
    void enqueue (int x) {
        S1.push (x);
    }

    int dequeue () {
        int x;
        if (S1.empty () && S2.empty()) {
            return -1;
        }

        if (S2.empty()) {
            while (! S1.empty () ) {
                S2.push (S1.top());
                S1.pop();
            }
        }
        x = S2.top();
        S2.pop();
        return x;
    }

    bool isEmpty() {
        return S1.empty () && S2.empty();
    }
};
```

DATE

Ans-5 (b) Given: $f(n) = 3n^2 + 4n \log n + 5n$

Show: $f(n) \in O(n^2)$

Proof: $f(n) \leq C \cdot n^2$

$3n^2 + 4n \log n + 5n \leq C \cdot n^2$

$3n^2 + 4n \log n + 5n \leq 5n^2 \quad \forall \quad n \geq 0$

~~Ans-5 (c)~~

~~Ans-6 (a)~~

Ans-6 (b) Function to return i^{th} element of a singly linked list.

```
int getElement(Node* head, int i) {
    if (head == nullptr || i<=0) {
        cout<< "Invalid index";
        return -1;
    }
    Node* temp = head;
    int count = 1;

    while (temp != nullptr && count<i) {
        temp = temp->next;
        count++;
    }

    if (temp == nullptr) {
        cout<< "Position out of range" << endl;
        return -1;
    }
    return temp->data;
}
```

DATE

Time complexity of accessing an element in array is O(1) and time complexity of accessing an element in singly linked list is O(n).

Ans-7 (b)

```
void deleteAtCursor(Node* cursor){
    if (cursor = nullptr) {
        cout << "List is empty" << endl;
        return;
    }

if (cursor -> next == cursor) {
    delete cursor;
    cursor = nullptr;
    return;
}

Node* temp = cursor;
while (temp -> next != cursor)
        temp = temp -> next;

temp -> next = cursor -> next;
Node* delNode = cursor;
cursor = cursor -> next;

delete delNode;
}
```

DATE

Ans-7 (a) Recursive function to count nodes of Binary Search Tree.

```
int countNodes( Node* root){
    if (root == nullptr)
        return 0;
    else
        return 1+ countNodes(root->left) +
                countNodes.(root->right);

}
```